PRINCETON UNIVERSITY PRESS

Chapter Title: C RANDOM NUMBERS AND WALKS

Book Title: Natural Complexity

Book Subtitle: A Modeling Handbook

Book Author(s): Paul Charbonneau

Published by: Princeton University Press

Stable URL: https://www.jstor.org/stable/j.ctt1vwmgzt.18

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

Princeton University Press is collaborating with JSTOR to digitize, preserve and extend access to *Natural Complexity*

JSTOR

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

C

RANDOM NUMBERS AND WALKS

C.1 Random and Pseudo-Random Numbers

A sequence of numbers is said to be *random* if the numerical value of each member in the set is entirely independent of the numerical value of the other members of the set. Once upon a time, I enlisted my then six-year-old son to roll a standard six-faced die 12 times in a row, twice so; the results were the following two sequences:

$$4-2-6-3-4-4-2-6-1-5-2-6,$$
$$6-6-1-2-3-3-5-6-3-6-2-3.$$

You may note that the second sequence does not include a single "4." This is not so surprising as one may think, considering that the probability of *not* rolling a "4" is $1-1/6=5/6$, so that the probability of not rolling a "4" 12 times in a row, is $(5/6)^{12}=0.112$. This is small, but certainly not astronomically so (unlike your odds of winning at the lottery, which are). If indeed each throw is entirely independent of the preceding throw, then the odds of obtaining exactly one of these sequences is $(1/6)^{12}=4.6\times10^{-10}$, which is in fact exactly the same as obtaining one

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

of the following two sequences, which most people would judge, incorrectly, to be far less probable:

$$1 - 2 - 3 - 4 - 5 - 6 - 1 - 2 - 3 - 4 - 5 - 6,$$

$$6 - 6 - 6 - 6 - 6 - 6 - 6 - 6 - 6 - 6 - 6 - 6.$$

Now, if you roll a die a very great many times (N, say), then you would expect to roll "1" $N/6$ times, "2" $N/6$ times also, and so on to "6." If you get different numbers, then you should really take a closer look at that die. For an unloaded die, every roll is independent of the others, and every one of the six possible outcomes is equiprobable. In other words, the die is a *generator* of random integers uniformly distributed in the interval [1, 6], and die-throwing is categorized as a stationary, memoryless random process.

How do you achieve the same thing on a computer? At first glance this may appear nonsensical, considering that a computer program is entirely deterministic; on a given architecture, an executable program will always return the same output upon being presented with the same input. This means that a computer program autonomously simulating successive throws of a die will always produce the same sequence. In other words, the result of the nth "throw" will be entirely determined by the state of the computer's memory following the $(n - 1)$th throw, this being true for all throws. Successive throws in the sequence are completely correlated; we could be no further from a memoryless random process.

The way out of this paradox is to accept the fact that successive throws will be perfectly correlated, but design our die-throwing algorithm so that, *over a long sequence of throws*,

- every throw value is equiprobable;
- there is no statistical correlation between successive throws; in other words, a "1" anywhere in the sequence is as likely to be followed by a "1," a "2," a "3," etc.

It is this *statistical uniformity* of the sequence that defines its random status, although the term "pseudo-random" is usually

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

preferred, to distinguish it from truly random sequences, such as die throws, coin flips, or radioactive decay.

C.2 Uniform Random Deviates

Many simulation codes listed in this book require either a random number generator which returns floating-point numbers uniformly distributed in some fixed interval, or integers distributed uniformly in some range $[0, N]$. Python's NumPy library contains such generators (and many others). Generic random number generators exist in most programming languages. The theoretical, arithmetical, statistical, and computational underpinnings of the generation of pseudo-random numbers are rather intricate and would fill many pages, but this would not be particularly useful here. For the purposes of working through this book, all you need to know is that pseudo-random number generators do exist, some are better than others, a few are downright crappy, and by now the truly objectionable among these have gone extinct.

As an example, what follows are two distinct ways to generate the computational equivalent of rolling a six-faced die:

```
import numpy as np
...
roll=np.random.random_integers(1,6)
...
```

Note that here, the upper and lower bounds given are inclusive (*unlike* the `range()` function controlling unconditional loops in Python), so that the above call will return 1, 2, 3, 4, 5, or 6 equiprobably. The other way is

```
import numpy as np
...
roll=np.random.choice([1,2,3,4,5,6])
...
```

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

Sometimes it is necessary to generate distinct sequences of random numbers, for example when testing different realizations of a stochastic process for the purposes of ensemble averaging; many such instances can be found in this book. The NumPy library includes a function named `numpy.random.seed()`, which allows the numerical value of the seed to be set for subsequent calls to any one of Python/NumPy's random number generators, by passing a specific integer value as an argument, for example `numpy.random.seed(1234)`.

C.3 Using Random Numbers for Probability Tests

The ability to generate random numbers uniformly distributed in the unit interval allows a simple numerical implementation of probability tests, in simulations involving stochastic rules. In the forest-fire model of chapter 6, for example, a tree on a lattice node can be ignited by lightning with probability p_{f}. For each such tree, the "decision" to ignite or not can be encapsulated in a one-line conditional statement:

```
if np.random.uniform() < p_f: # lightning strikes (maybe)
```

Since successive draws of the random number r (as produced by `np.random.uniform()`$\equiv r$) are uniformly distributed in [0, 1], then for $p_{\mathrm{f}} = 10^{-5}$ (say), on average one draw in 10^5 will satisfy $r < p_{\mathrm{f}}$. Consequently, on average one in every 10^5 trees will be ignited by lightning at each temporal iteration. If the lattice is very large and contains a number of trees $\gg p_{\mathrm{f}}^{-1}$, then many trees will be ignited at each iteration; in the opposite situation, ignition events will be separated in time, with the wait time between successive lightning strikes distributed exponentially in the regime $p_{\mathrm{f}} \ll p_{\mathrm{g}}$.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

The above procedure effectively draws pseudo-random numbers from a PDF (see appendix B) of the form

$$f(x) = \begin{cases} 1, & 0 < x \leq 1, \\ 0 & \text{otherwise}, \end{cases} \tag{C.1}$$

which satisfies the normalization condition (B.5). However, the discrete PDF for a sequence of N pseudo-random numbers, constructed following the procedure described in appendix B.1, will converge to equation (C.1) only in the limit $N \to \infty$. Figure C.1 illustrates this convergence, for N-member sequences of pseudo-random numbers with N increasing from 300 in panel A to 300,000 in D. In all cases, the bin size is $b = 0.05$, so that $M = 20$ bins are required to cover the interval. This PDF being normalized, the expected value of every bin, in the limit $N \to \infty$, is unity. Clearly, fluctuations about this expected value decrease rapidly as N increases. This decrease can be quantified by computing the *root-mean-squared* deviation about the expected value:

$$\sigma = \left(\frac{1}{M} \sum_{m=1}^{M} (h_m - 1)^2 \right)^{1/2}. \tag{C.2}$$

The dotted lines in figure C.1 indicate the range $\pm\sigma$ about the expected value of unity. As with any stationary, memoryless, random process, σ varies as $1/\sqrt{N}$.

C.4 Nonuniform Random Deviates

In some situations it can be useful, or even necessary, to produce sequences of pseudo-random numbers extracted from nonuniform probability distributions. Python's NumPy library contains many functions producing various common distributions of random deviates. If you only have access to a function providing uniform random deviates, it is still possible to generate other types

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

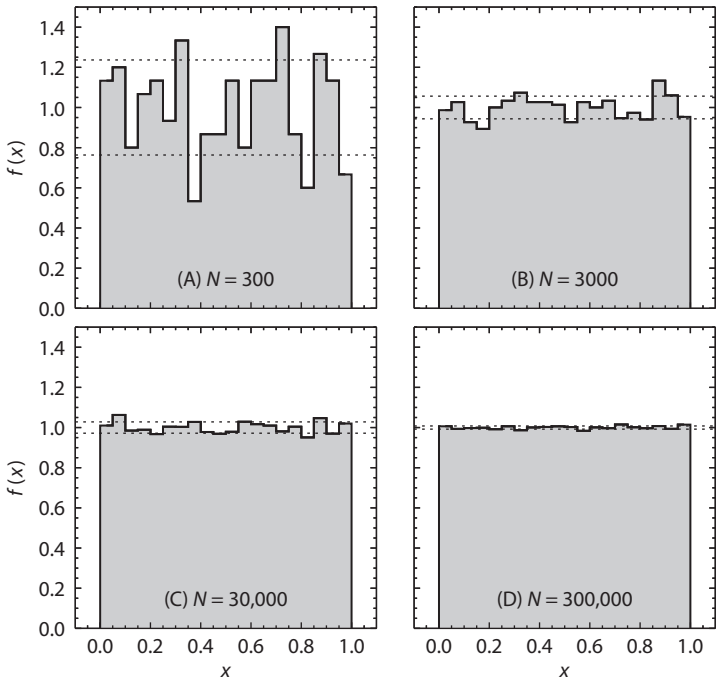

Figure C.1. Probability density functions constructed from sequences of N random numbers uniformly distributed in the interval $[0, 1]$, with N increasing by successive factors of 10 going from panels A through D. The horizontal dotted lines indicate the range $\pm\sigma$ about the expected value $f(x) = 1$.

of distributions, through a technique known as the *transformation method*. What follows states only a few useful results. In all cases, r is a random deviate extracted from a uniform unit distribution, $r \in [0, 1[$, and x is the sought deviate from another distribution.

To get uniform random deviates in the range $[a, b]$, a simple linear rescaling does the job:

$$x = a + (b - a)r, \quad r \in [0, 1], \quad x \in [a, b]. \tag{C.3}$$

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

In this case, the mean of the distribution is $\langle x \rangle = (a + b)/2$. This can also be achieved by the Python/NumPy function call `x=numpy.random.uniform(a,b)`.

An exponential deviate in the range $x \in [0, \infty]$ is given by

$$x = -\lambda \ln r, \quad r \in [0, 1], \quad x \in [0, \infty]. \tag{C.4}$$

Note that for $r \in [0, 1]$, $\ln(r) < 0$; the minus sign in equation (C.4) is important, don't forget it! Here, the parameter λ sets the scale of the exponential falloff; smaller values of λ give a more steeply peaked distribution, and larger values a flatter exponential distribution. In all cases, the mean value of the distribution, as given by equation (B.6), is $\langle x \rangle = \lambda$, even though the most probable value is zero.

Under Python's NumPy library, the function call `x=numpy.random.exponential(scale)` produces such exponential deviates, but do note here that `scale`$\equiv 1/\lambda$ in the above expressions.

For a power-law PDF (see equation (B.7)) normalized to unity in the range $[1, \infty]$, the required transformation is

$$x = r^{1/(1-\alpha)}, \quad r \in [0, 1], \quad x \in [1, \infty], \quad \alpha > 1. \tag{C.5}$$

The artificial data used to generate the PDFs in figure B.3 were generated in this manner.

The function `numpy.random.power(a)` in Python's NumPy library can be used to produce power-law deviates, but *beware*, its argument `a` corresponds to $1 - \alpha$ under the power-law definition used throughout this book; see equation (B.7).

Another useful PDF is the Gaussian (or normal) distribution.[1] Gaussian distributions of random deviates can be easily generated

[1]Statistical theory would state that in the absence of cheating and with fair grading, and in the limit of infinite class size (ackpht!), the PDF of my midterm exam grades plotted in figure B.2 should be a Gaussian centered on the class average!

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

through the Box–Muller transformation, which produces *two* Gaussian deviates x_1, x_2 from *two* uniform deviates r_1, r_2 via the relations

$$x_1 = \sqrt{-2\ln r_1}\cos(2\pi r_2), \quad x_2 = \sqrt{-2\ln r_1}\sin(2\pi r_2),$$
$$r_1, r_2 \in [0, 1], \quad x \in [-\infty, \infty]. \tag{C.6}$$

The deviates so generated fill a Gaussian distribution of zero mean and unit variance. If the deviates need to be centered about a nonzero mean value (x_0, say) with a standard deviation $\sigma \neq 1$, then they should be rescaled as

$$g_1^* = x_0 + \sigma \times g_1, \qquad g_2^* = x_0 + \sigma \times g_2. \tag{C.7}$$

The function call `g=numpy.random.normal(x0,sigma)` in Python's NumPy library can be used to produce Gaussian deviates of mean value x_0 and standard deviation σ.

C.5 The Classical Random Walk

A random walk describes the changing position of an agent taking successive steps, all of the same length s, but oriented randomly, in the memoryless sense that, not only is the orientation of step n random, but it is also entirely independent of the orientations of previous steps.

Consider first a 1-D random walk, where the displacement is constrained to lie along a line (think of a very narrow road with high fences on both sides, or a very long doorless corridor within a building). The displacement at step n, measured with respect to some starting position, is denoted by D_n, and the two equiprobable steps are $s_n = \pm 1$. By definition we can write

$$D_n = D_{n-1} + s_n, \quad n = 0, 1, 2, 3 \ldots. \tag{C.8}$$

Note already that the total distance walked, $n \times s$, is not the same as the displacement measured from the starting location; two steps to the right followed by two to the left add up to

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

zero displacement, even though four steps have been taken. The squared displacement is then

$$D_n^2 = (D_{n-1} + s_n)^2 = D_{n-1}^2 + s^2 + 2D_{n-1}s_n. \tag{C.9}$$

Now consider a group of M agents, all starting at the same position and each engaging in a (collisionless) random walk. Introduce now the *ensemble average*, denoted by the brackets $\langle \ldots \rangle$, defined over this whole group:

$$\langle x \rangle = \frac{1}{M} \sum_{m=1}^{M} x(m). \tag{C.10}$$

Under this notation, the quantity $\langle D_n \rangle$ can be interpreted as the average displacement of the group as a whole. Averaging is a linear operator, in the sense that

$$\langle x + y \rangle = \langle x \rangle + \langle y \rangle, \qquad \langle ax \rangle = a\langle x \rangle, \tag{C.11}$$

where a is any numerical coefficient. Because of this linearity, applying our averaging operator to equation (C.9) yields

$$\begin{aligned} \left\langle D_n^2 \right\rangle &= \left\langle D_{n-1}^2 + s^2 + 2D_{n-1}s_n \right\rangle \\ &= D_{n-1}^2 + \left\langle s^2 \right\rangle + 2\langle D_{n-1}s_n \rangle. \end{aligned} \tag{C.12}$$

If no communication or interaction takes place between agents and consequently they have no way to get in step with one another, then for a large enough group of agents, $\langle s_n \rangle = 0$ since right- and left-directed steps are equiprobable. Moreover, for a memoryless process, the distribution of steps ± 1 at iteration n is entirely uncorrelated to the distribution of displacements D_{n-1} at the prior step. This implies

$$\langle D_{n-1}s_n \rangle = \langle D_{n-1} \rangle \langle s_n \rangle = 0. \tag{C.13}$$

This result is critical for all that follows. Equation (C.12) now becomes

$$\left\langle D_n^2 \right\rangle = \left\langle D_{n-1}^2 \right\rangle + s^2. \tag{C.14}$$

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

Setting $D_0 = 0$ without loss of generality, we have

$$\left\langle D_1^2 \right\rangle = s^2, \tag{C.15}$$

$$\left\langle D_2^2 \right\rangle = \left\langle D_1^2 \right\rangle + s^2 = 2s^2, \tag{C.16}$$

$$\left\langle D_3^2 \right\rangle = \left\langle D_2^2 \right\rangle + s^2 = 3s^2, \tag{C.17}$$

$$\vdots \tag{C.18}$$

and so, after n steps,

$$\left\langle D_n^2 \right\rangle = ns^2. \tag{C.19}$$

If the (discrete) variable n is interpreted as a temporal iteration, this expression indicates that the mean quadratic displacement increases linearly with time, so that

$$\sqrt{\left\langle D_n^2 \right\rangle} = s\sqrt{n}. \tag{C.20}$$

This is called the *root-mean-squared* displacement. It is important to understand that even though this increases with time, the mean displacement $\langle D_n \rangle$ vanishes at all times. The distinction is easier to understand by simulating a great many random walks and constructing distribution functions for the positions of the walkers. An example is shown in figure C.2, for a simulation involving 1000 1-D random walkers, all starting at $x = 0$. The distributions are constructed and plotted after 1000, 3000, 10,000, and 30,000 steps, as color coded. It is clear from these plots that the mean of each distribution, i.e., $\langle D_n \rangle$, always remains very close to zero, even after 30,000 steps, and all that time the most probable displacement, coinciding with the peak value of the PDF, is also essentially zero. Yet, equally obviously, the distribution spreads outward with time, so that the probability of finding a large displacement, either positive or negative, increases with time.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

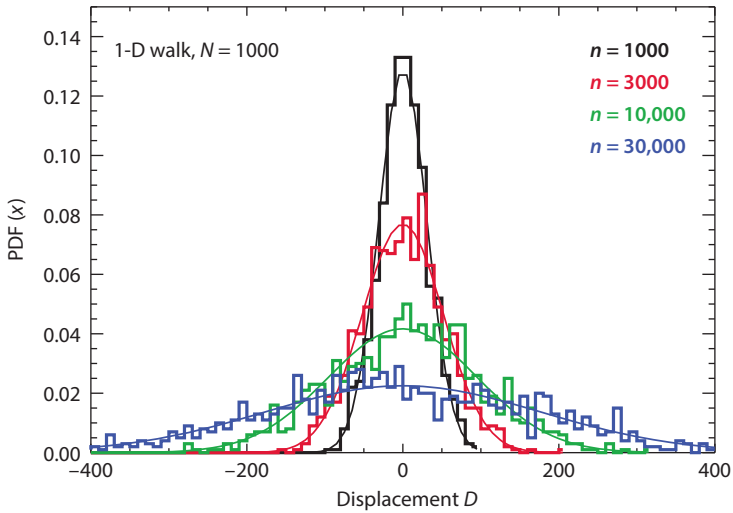

Figure C.2. Temporal spreading of the distribution of 1000 1-D random walkers, all originally located at $x = 0$. The thin lines are the Gaussian best fit to the distribution data, color coded correspondingly.

The colored thin lines are least-squares fits to these distributions, computed by adjusting the parameter σ of the Gaussian PDF

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-x^2}{\sigma^2}\right). \tag{C.21}$$

This parameter is a measure of the width of the Gaussian distribution (the full width at half-maximum is $1.176 \times \sigma$, with 68.3% of all measurements contained within $\pm\sigma$ of the mean). Here, σ can be shown to increase with time as $\sqrt{n}$, which is the same pseudo-temporal dependence arrived at when directly computing the root-mean-squared displacement (equation (C.20)).

All of these results carry over to random walks in more than one spatial dimension. The displacement $\boldsymbol{D}_n$ and step $\boldsymbol{s}_n$ become

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

vector quantities, and equation (C.8) must be replaced by

$$\boldsymbol{D}_n = \boldsymbol{D}_{n-1} + \boldsymbol{s}_n, \quad n = 1, 2, 3 \ldots, \tag{C.22}$$

where the step $\boldsymbol{s}_n$ still has unit length but is oriented randomly in space. The mean square displacement at step n becomes

$$\begin{aligned} D_n^2 &= (\boldsymbol{D}_{n-1} + \boldsymbol{s}_n) \cdot (\boldsymbol{D}_{n-1} + \boldsymbol{s}_n) \\ &= D_{n-1}^2 + s^2 + 2\boldsymbol{D}_{n-1} \cdot \boldsymbol{s}_n. \end{aligned} \tag{C.23}$$

Once again, if the step orientation is truly random and uncorrelated to the displacement vector at the prior step, then averaged over a large ensemble of walkers, $\langle \boldsymbol{D}_{n-1} \cdot \boldsymbol{s}_n \rangle = 0$. Why this is so is exemplified in figure C.3, showing the first 18 steps of a 2-D random walk, beginning at $(x, y) = (0, 0)$, with a step length $|\boldsymbol{s}| = 1$, and the thick red line indicating the rms displacement vector after the 18th step, i.e., $\boldsymbol{D}_{18}$. This vector makes an angle θ_{18} with respect to the x-axis of a Cartesian coordinate system centered on $(0, 0)$, as indicated by the dotted lines. The 19th step will land the walker somewhere on the green circle of unit radius centered on the walker's position at the 18th step. Where on this circle the walker will actually land is entirely random, i.e., the angle α_{19} of its next step with respect to a local coordinate system centered on $\boldsymbol{D}_{18}$ can be anything between 0 and 2π, equiprobably. Now, the angle between the displacement vector $\boldsymbol{D}_{18}$ and $\boldsymbol{s}_{19}$ will be given by $\boldsymbol{s}_{19} - \boldsymbol{D}_{18}$, so that

$$\boldsymbol{D}_{18} \cdot \boldsymbol{s}_{19} = D_{18} s_{19} \cos(\alpha_{19} - \theta_{18}). \tag{C.24}$$

At this point in the walk the angle θ_{18} is already set, at some value between 0 and 2π; whereas α_{19} is drawn randomly from a uniform distribution spanning $[0, 2\pi]$. Trigonometric functions being periodic, it is as if the angle $\alpha_{19} - \theta_{18}$ were also drawn from a uniform distribution spanning $[0, 2\pi]$; its cosine is therefore as likely to turn out positive as negative, both identically distributed, which ensures that an ensemble average of the above scalar product will always vanish—even though it almost never would

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

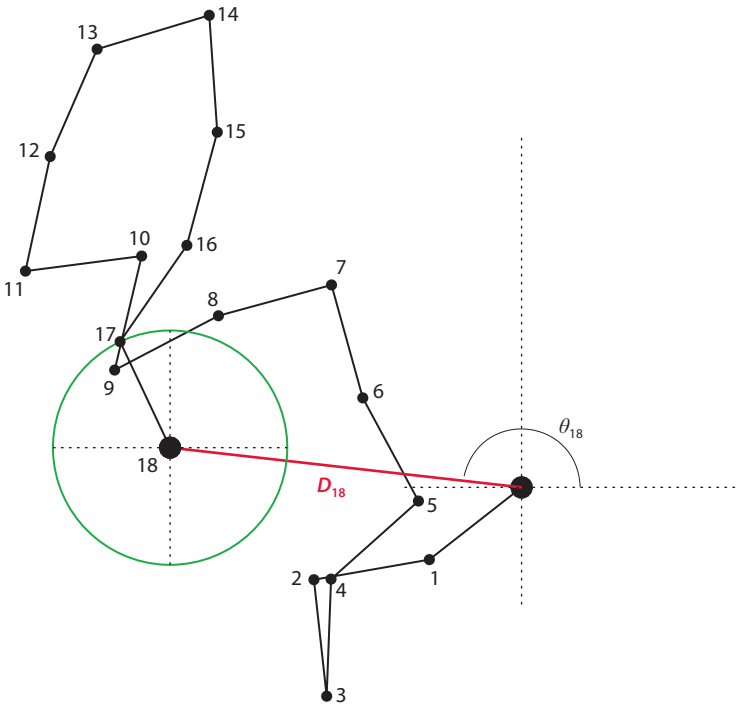

Figure C.3. The first 18 steps of a 2-D random walk in the plane, with unit-length step $\boldsymbol{s}_n$. The numbered solid dots indicate the successive positions of the walker. The 19th step will land somewhere on the green circle centered on 18. Where it will land on that circle, i.e., the spatial orientation of that 19th step, is entirely independent of the length and orientation of the current displacement vector $\boldsymbol{D}_{18}$ (red line segment).

for a single walker. The same reasoning would also hold if both angles α_{19} and θ_{18} were drawn *independently* from their allowed range. This evidently also holds for any step n, and leads to the conclusion that the ensemble average $\langle \boldsymbol{D}_{n-1} \cdot \boldsymbol{s}_n \rangle = 0$. Therefore, the ensemble average of equation (C.23) becomes

$$\left\langle D_n^2 \right\rangle = \left\langle D_{n-1}^2 \right\rangle + s^2, \tag{C.25}$$

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

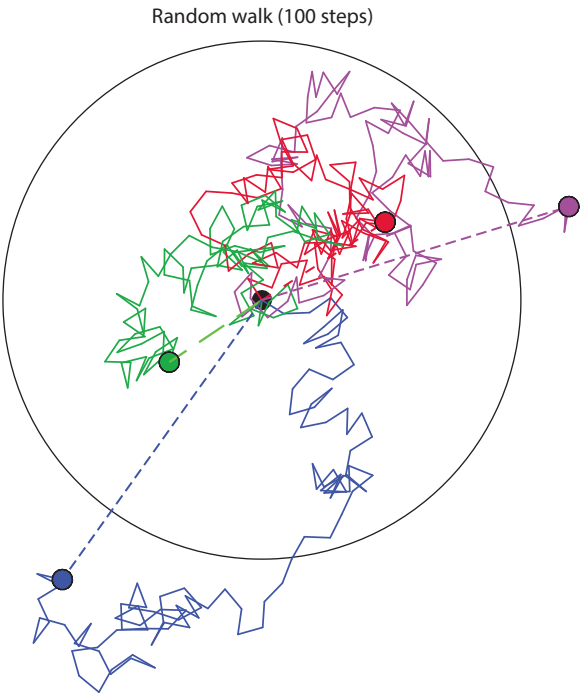

Figure C.4. Four 2-D random walks of 100 steps each. The starting point is indicated by a black dot at the center, and the circle indicates a displacement $D \equiv \sqrt{\boldsymbol{D} \cdot \boldsymbol{D}} = \sqrt{n} = 10$.

just as in the case of the 1-D random walk (cf. equation (C.14)). Everything else proceeds as before and leads again to equation (C.19). This is a truly remarkable property of random walks: no matter the dimensionality, the root-mean-squared displacement always increases as $\sqrt{n}$.

Figure C.4 illustrates a few 2-D random walks, each over 100 steps, with the circle drawn at the radius corresponding to the root-mean-squared displacement, $R = \sqrt{\langle D_{100} \rangle} = 10s$. This figure highlights, once again, the fact that the rms displacement is a statistical measure, obtained from an ensemble average over a very large number of walkers; the displacement of a given

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

individual walker can deviate substantially from equation (C.25), while remaining bound in $[0, n \times s]$.

C.6 Random Walk and Diffusion

The gradual spreading observed in the distribution of random walkers in figure C.2 is prototypical of *diffusive processes*, and this turns out to be more than a mere visual analogy.

Let's stick to 1-D random walks and consider what happens at some arbitrary position x_0; only walkers within the range $x_0 - |s| < x < x_0 + |s|$ have a chance to cross x_0 at the next step, but then again, only if they happen to step in the required direction ($s = +1$ for walkers in $x_0 - |s| < x < x_0$, and $s = -1$ for those in $x_0 < x < x_0 + |s|$). Both stepping directions being equiprobable, on average only half the walkers on each side will then cross x_0. Denote by $\delta N(x_0)$ the net number of walkers crossing x_0 from the right to the left. This quantity will be given by

$$\delta N(x_0) = \tfrac{1}{2} N(x_0 - |s|) - \tfrac{1}{2} N(x_0 + |s|) \tag{C.26}$$

(a negative value for δN would then mean that the net flow of walkers is from left to right). Now let $N(x_0, t)$ be the number of walkers standing somewhere in the full interval $x_0 - |s| < x < x_0 + |s|$ at time t. That number, at time $t + \Delta t$, will then be given by $N(x_0, t)$ plus the net number having entered from the left side at $x_0 - |s|$, minus the number having walked out to the right across the right boundary $x_0 + |s|$. Evaluating equation (C.26) at $x_0 - |s|$ and $x_0 + |s|$, instead of just x_0, then leads to

$$\begin{aligned} N(x, t + \Delta t) &= N(x, t) + \delta N(x - s) - \delta N(x + s) \\ &= N(x, t) + \left(\left(\tfrac{1}{2} N(x - 2s) - \tfrac{1}{2} N(x)\right)\right. \\ &\quad \left. - \left(\tfrac{1}{2} N(x) - \tfrac{1}{2} N(x + 2s)\right)\right) \\ &= N(x, t) + \tfrac{1}{2} \left(N(x + 2s) - 2N(x) + N(x - 2s)\right), \end{aligned} \tag{C.27}$$

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

where we have dropped the "0" index on x and the absolute value on s to lighten the notation. Dividing the right- and left-hand sides of this expression by Δt and rearranging terms, we get

$$\frac{N(x, t + \Delta t) - N(x, t)}{\Delta t} = \frac{1}{2}\left(\frac{(2s)^2}{\Delta t}\right) \times \left(\frac{N(x + 2s) - 2N(x) + N(x - 2s)}{(2s)^2}\right). \quad \text{(C.28)}$$

Note that both the numerator and denominator of the right-hand side have been multiplied by the quantity $2s^2$. This mathematically legal, but by all appearances arbitrary, manoeuver was carried out so that the quantity within the second set of parentheses on the right-hand side is identical to a second-order centered finite difference formula for the second derivative of N with respect to x, with a spatial discretization increment $2s$. The term on the left-hand side is a first-order forward difference formula for the time derivative of N, with time step Δt. If these interpretations are accepted, then equation (C.28) can be viewed as a finite difference discretization of the partial differential equation

$$\frac{\partial N(x, t)}{\partial t} = D\frac{\partial^2 N(x, t)}{\partial x^2}, \quad \text{(C.29)}$$

where D is a *diffusion coefficient*, here given by

$$D = \frac{1}{2}\frac{(2s)^2}{\Delta t}. \quad \text{(C.30)}$$

Equation (C.29) is the well-known classical linear diffusion equation, which represents a macroscopic description of a random walk; it also describes the spreading of perfume (or other smell) in a room where the air is at rest, the slow diffusive mixing of cream in a coffee that is *not* being stirred, as well as a host of other common mixing and dilution processes. Equation (C.29) holds, provided the flux of the diffusing quantity is proportional to the (negative) concentration gradient of the diffusive substance,

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms

which is known as linear (or Fickian) diffusion. The physical link with the random walk arises from the random motion of perfume or cream molecules, continuously colliding with molecules making up the background fluid (air or coffee). Here is one of those instances where understanding the microscopic behavior, namely, the random walk, allows the macroscopic behavior, i.e., diffusion, to be calculated.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:01 UTC
All use subject to https://about.jstor.org/terms